Experiment No.9
Development of a ReactJS Application with API Integration and Search Functionality
Date of Performance: 25/03/2026
Date of Submission: 25/03/2026

Aim: To develop a ReactJS application with API integration and search functionality.

Objective: To build a dynamic web application using ReactJS that fetches data from external APIs, implements search functionality, and updates the user interface efficiently using component-based architecture.

Theory:
ReactJS is a popular JavaScript library used for building dynamic and interactive user interfaces, especially for single-page applications. It follows a component-based architecture, where the UI is divided into reusable components, making development more efficient and organized. React also uses a virtual DOM, which improves performance by updating only the necessary parts of the webpage instead of reloading the entire page.

In a React application, data can be fetched from external APIs using methods like fetch or axios, allowing real-time data integration. Search functionality can be implemented by capturing user input and filtering the displayed data dynamically. React efficiently updates the UI based on changes in state and props, ensuring a smooth and responsive user experience. This approach helps in building scalable and high-performance web applications.

Implementation:

```
import React, { useState, useEffect } from "react";

import axios from "axios";


function App() {

  const [users, setUsers] = useState([]);

  const [search, setSearch] = useState("");
```

```
// Fetch API Data
useEffect(() => {
  axios.get("https://jsonplaceholder.typicode.com/users")
    .then(res => setUsers(res.data))
    .catch(err => console.log(err));
}, []);

// Filter Data
const filteredUsers = users.filter(user =>
  user.name.toLowerCase().includes(search.toLowerCase())
);

return (
  <div style={{ textAlign: "center", padding: "20px" }}>
    <h2>User Search App</h2>

    {/* Search Box */}
    <input
      type="text"
      placeholder="Search user..."
      value={search}
      onChange={(e) => setSearch(e.target.value)}
      style={{ padding: "10px", width: "250px" }}
    />

    {/* User List */}
```

```
      <ul style={{ listStyle: "none", marginTop: "20px" }}>
        {filteredUsers.map(user => (
          <li key={user.id} style={{ padding: "10px", borderBottom: "1px solid #ccc" }}>
            <strong>{user.name}</strong><br/>
            {user.email}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

User Search App

Search user...

Leanne Graham
Sincere@april.biz

Ervin Howell
Shanna@melissa.tv

Clementine Bauch
Nathan@yesenia.net

Patricia Lebsack
Julianne.OConner@kory.org

Chelsey Dietrich
Lucio_Hettinger@annie.ca

Mrs. Dennis Schulist
Karley_Dach@jasper.info

Kurtis Weissnat
Telly.Hoeger@billy.biz

Nicholas Runolfsdottir V
Sherwood@rosamond.me

Glenna Reichert
Chaim_McDermott@dana.io

User Search App

leanne

Leanne Graham
Sincere@april.biz

Conclusion:

Building a dynamic web application using **ReactJS** enables efficient data handling and responsive user interfaces through its component-based architecture. By fetching data from external **APIs**, the application provides real-time and relevant information. The implementation of **search functionality** enhances user interaction by allowing quick data filtering. Overall, ReactJS ensures smooth UI updates and improved performance, making the application interactive, scalable, and user-friendly.